INTRODUCTION AUX BASES DE DONNÉES · L2 · TD3

# SQL : les rappels du TD3

Créer des tables, les remplir, les interroger : l'essentiel pour traiter les quatre exercices avec PostgreSQL.

```sql
SELECT * FROM rappels WHERE td = 3;
```

REPÈRES

Vue d’ensemble du TD3

# Trois gestes pour quatre exercices

## 1

### Créer les tables

`CREATE TABLE`

Types, contraintes, clés primaires et étrangères.

Pâtisserie · Collège · Communes · Bibliothèque

## 2

### Remplir les tables

`INSERT INTO`

Écrire les valeurs, insérer dans le bon ordre.

Pâtisserie · Communes

## 3

### Interroger

`SELECT`

Filtrer, trier, joindre, regrouper.

Pâtisserie, questions 3(a) à 3(i)

Le point de départ est toujours un MLD, comme ceux des TD1 et TD2 : ces rappels commencent là où il s’arrête.

REPÈRES

Pour : les quatre exercices

# Le script SQL, de haut en bas

```sql
-- 1. Repartir de zéro
DROP TABLE IF EXISTS seance;
DROP TABLE IF EXISTS film;

-- 2. Créer les tables
CREATE TABLE film (…);
CREATE TABLE seance (…);

-- 3. Les remplir
INSERT INTO film VALUES (…);

-- 4. Les interroger
SELECT * FROM film;
```

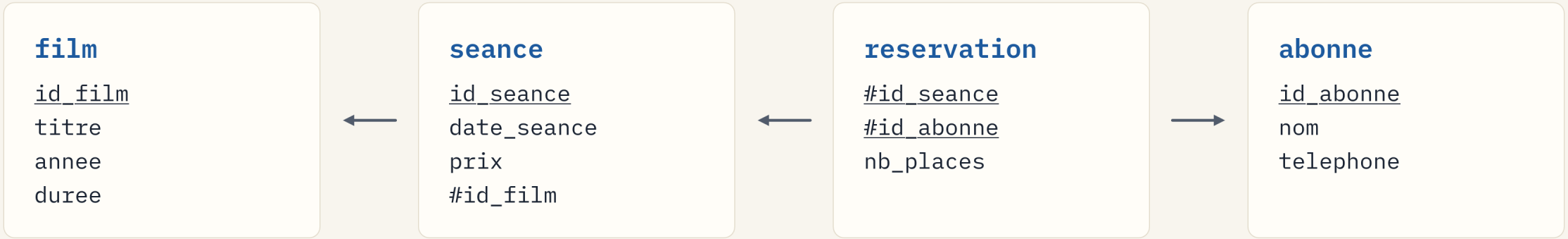
| | |
|---|---|
| `;` | termine chaque instruction |
| `'La Haine'` | apostrophes simples : un texte ou une date |
| `"film"` | guillemets doubles : un nom de table ou de colonne, jamais un texte |
| `-- note` | commentaire, jusqu'à la fin de la ligne |
| `select = SELECT` | casse libre pour les mots-clés ; les majuscules sont une convention |
| `date_seance` | noms sans accent ni espace, hors mots réservés comme order, group ou user |

REPÈRES

Fil rouge des exemples

# La base d’exemple : un cinéma

Un cinéma projette des films lors de séances ; ses abonnés réservent des places pour ces séances.




Souligné : clé primaire · # : clé étrangère · chaque flèche part d’une clé étrangère et pointe vers la clé primaire qu’elle référence.

---

Tous les exemples portent sur cette base, jamais sur celles des exercices : à vous de transposer.

1 · CRÉER LES TABLES

Pour : les quatre exercices

# Du schéma relationnel à CREATE TABLE

**DANS LE MLD** `seance (id_seance, date_seance, prix, #id_film)`

```
CREATE TABLE seance (
    id_seance   INTEGER PRIMARY KEY,
    date_seance DATE NOT NULL,
    prix        NUMERIC(5,2) NOT NULL,
    id_film     INTEGER NOT NULL REFERENCES film(id_film)
);
```

La relation

`CREATE TABLE nom (…);`

Chaque attribut

`nom TYPE contraintes,`

La clé primaire, soulignée

`PRIMARY KEY`

La clé étrangère, notée #

`REFERENCES table(colonne)`

---

Une virgule sépare les lignes, sauf après la dernière ; le point-virgule ferme l’instruction.

Pour : les quatre exercices

# Les types de colonnes

| TYPE | POUR STOCKER | VALEUR ÉCRITE |
|---|---|---|
| `INTEGER` | un entier : quantité, année, effectif | 42 |
| `NUMERIC(5,2)` | un décimal exact : un prix (5 chiffres, dont 2 décimales) | 8.50 |
| `VARCHAR(50)` | un texte d'au plus 50 caractères : nom, libellé | `'Dupont'` |
| `TEXT` | un texte long, sans limite : description | `'Roman policier'` |
| `DATE` | une date | `'2026-11-12'` |
| `TIME` | une heure | `'14:30'` |
| `TIMESTAMP` | une date et une heure | `'2026-11-12 14:30'` |
| `BOOLEAN` | vrai ou faux | TRUE |
| `SERIAL` | un identifiant numéroté automatiquement : 1, 2, 3… | rien à saisir |

LE PIÈGE COMMUN AUX QUATRE EXERCICES

# Un code n’est pas un nombre.

Téléphone, INE, ISBN, code postal, code INSEE : on ne calcule jamais dessus. Ce sont des textes, déclarés en VARCHAR ou CHAR et écrits entre apostrophes.

`01001 → 1001`

Rangé dans un INTEGER, le code perd son zéro initial : plus de 3 000 communes sont concernées.

`2A004 → erreur`

Rangé dans un INTEGER, un code corse est refusé : la lettre A n’est pas un chiffre.

Pour : les quatre exercices

# Les contraintes de colonne

| CONTRAINTE | CE QU'ELLE GARANTIT | EXEMPLE |
|---|---|---|
| `PRIMARY KEY` | identifie la ligne : unique et jamais vide | `id_film INTEGER PRIMARY KEY` |
| `NOT NULL` | la colonne est toujours renseignée | `titre VARCHAR(100) NOT NULL` |
| `UNIQUE` | deux lignes n'ont jamais la même valeur | `telephone VARCHAR(10) UNIQUE` |
| `CHECK (…)` | la valeur respecte une condition | `prix NUMERIC(5,2) CHECK (prix >= 0)` |
| `DEFAULT …` | une valeur est fournie si l'on n'en donne pas | `date_seance DATE DEFAULT CURRENT_DATE` |
| `REFERENCES` | la valeur existe dans une autre table | `id_film INTEGER REFERENCES film(id_film)` |

Une contrainte suit le type, sur la ligne de sa colonne. Une condition peut aussi limiter les valeurs à une liste :

```
etat VARCHAR(10) CHECK (etat IN ('neuf', 'bon', 'abîmé'))
```

Pour : les quatre exercices

# La clé étrangère

```sql
CREATE TABLE film (
    id_film INTEGER PRIMARY KEY,
    titre   VARCHAR(100) NOT NULL
);

CREATE TABLE seance (
    id_seance INTEGER PRIMARY KEY,
    id_film   INTEGER NOT NULL REFERENCES film(id_film)
);
```

Variante, en fin de table :

```sql
FOREIGN KEY (id_film) REFERENCES film(id_film)
```

### Le même type que la clé visée

id_film est un INTEGER des deux côtés.

### La table visée existe déjà

Créer film avant seance ; supprimer seance avant film.

### NOT NULL si le lien est obligatoire

Sans lui, une séance pourrait n’avoir aucun film.

### Ce que PostgreSQL refuse ensuite

Une séance dont le film n’existe pas ; la suppression d’un film qui a encore des séances.

Pour : les associations plusieurs à plusieurs

# La table d’association

```
CREATE TABLE reservation (
    id_seance INTEGER REFERENCES seance(id_seance),
    id_abonne INTEGER REFERENCES abonne(id_abonne),
    nb_places INTEGER NOT NULL CHECK (nb_places > 0),
    PRIMARY KEY (id_seance, id_abonne)
);
```

### Deux clés étrangères

Une vers chaque table reliée.

### Une clé primaire composée

Le couple (id_seance, id_abonne), déclaré en fin de table. Deux PRIMARY KEY écrits sur deux colonnes sont refusés.

### L’attribut de l’association

nb_places devient une colonne ordinaire.

**À SURVEILLER** Si le même couple peut revenir plusieurs fois (un historique, plusieurs dates), il ne suffit plus comme clé : ajouter la date à la clé, ou donner à la table son propre identifiant.

# L’insertion des lignes : INSERT INTO

```
INSERT INTO film (id_film, titre, annee, duree) VALUES
    (1, 'La Haine', 1995, 98),
    (2, 'Les Visiteurs', 1993, 107);
INSERT INTO abonne (id_abonne, nom, telephone) VALUES
    (3, 'Nguyen', NULL);
INSERT INTO seance (id_seance, date_seance, prix, id_film) VALUES
    (1, '2026-11-02', 8.50, 1);
```

Un texte, entre apostrophes

`'La Haine'`

Une date, année-mois-jour

`'2026-11-02'`

Un nombre, avec un point décimal

`8.50`

Une valeur absente

`NULL`

Les parents d’abord : le film 1 doit exister avant la séance qui le référence.

Par défaut, '12/11/2026' serait lu comme le 11 décembre : écrire les dates année-mois-jour.

# Du fichier CSV aux INSERT

Le fichier des communes : environ 35 000 lignes et 62 colonnes, séparées par des virgules ; la première ligne nomme les colonnes.

| DANS LE FICHIER | DANS L'INSERT | POURQUOI |
|---|---|---|
| `01001` | `'01001'` | un code est un texte : en INTEGER, il devient 1001 |
| `2A004` | `'2A004'` | les codes corses contiennent une lettre |
| `L'Abergement-Clémenciat` | `'L''Abergement-Clémenciat'` | dans un texte, l'apostrophe se double |
| un champ vide | `NULL` | une valeur absente, sans apostrophes |
| `46.151` | `46.151` | un nombre : point décimal, sans apostrophes |

Dans un tableur, importer les colonnes de codes au format Texte : sinon 01001 s'affiche 1001.

Choisir des lignes cohérentes : chaque ligne « enfant » pointe vers une ligne « parent » déjà insérée.

# Les données de l'exemple

### film

| id_film | titre | annee | duree |
|---|---|---|---|
| 1 | La Haine | 1995 | 98 |
| 2 | Les Visiteurs | 1993 | 107 |
| 3 | Le Dîner de cons | 1998 | 80 |
| 4 | Les Choristes | 2004 | 97 |
| 5 | Intouchables | 2011 | 112 |
| 6 | Le Grand Bain | 2018 | 122 |

### abonne

| id_abonne | nom | telephone |
|---|---|---|
| 1 | Benali | 0639980101 |
| 2 | Dupont | 0639980102 |
| 3 | Nguyen | NULL |
| 4 | Moreau | 0639980104 |

### seance

| id_seance | date_seance | prix | id_film |
|---|---|---|---|
| 1 | 2026-11-02 | 8.50 | 1 |
| 2 | 2026-11-02 | 6.00 | 3 |
| 3 | 2026-11-04 | 8.50 | 5 |
| 4 | 2026-11-06 | 8.50 | 1 |
| 5 | 2026-11-07 | 10.00 | 5 |
| 6 | 2026-11-08 | 6.00 | 2 |
| 7 | 2026-11-09 | 8.50 | 4 |

### reservation

| id_seance | id_abonne | nb_places |
|---|---|---|
| 1 | 1 | 2 |
| 1 | 2 | 1 |
| 2 | 1 | 4 |
| 2 | 3 | 2 |
| 3 | 2 | 3 |
| 3 | 4 | 2 |
| 4 | 3 | 1 |
| 5 | 1 | 2 |
| 5 | 4 | 5 |
| 6 | 2 | 2 |

Pour : Pâtisserie, question 3

# L’anatomie d’un SELECT

Seuls SELECT et FROM sont obligatoires ; les clauses s’écrivent toujours dans cet ordre.

| ORDRE D’ÉCRITURE | RÔLE | ORDRE D’ÉVALUATION |
|---|---|---|
| `SELECT colonnes` | choisit ce qui est affiché | **5** |
| `FROM tables` | désigne les tables et les relie avec JOIN | **1** |
| `WHERE condition` | garde les lignes qui vérifient la condition | **2** |
| `GROUP BY colonnes` | réunit les lignes en groupes | **3** |
| `HAVING condition` | garde les groupes qui vérifient la condition | **4** |
| `ORDER BY colonnes` | trie le résultat | **6** |
| `LIMIT n` | ne garde que les n premières lignes | **7** |

WHERE agit avant les regroupements : un COUNT ou un SUM n’y a pas sa place.

Un alias créé dans SELECT est connu d’ORDER BY, pas de WHERE ni de HAVING.

# Le choix des colonnes : SELECT

### Toutes les colonnes : *

```
SELECT *
FROM abonne;
```

RÉSULTAT

| id_abonne | nom | telephone |
|---|---|---|
| 1 | Benali | 0639980101 |
| 2 | Dupont | 0639980102 |
| 3 | Nguyen | NULL |
| 4 | Moreau | 0639980104 |

### Sans doublon : DISTINCT

```
SELECT DISTINCT prix
FROM seance;
```

RÉSULTAT

| prix |
|---|
| 6.00 |
| 10.00 |
| 8.50 |

### Renommer, calculer : AS

```
SELECT titre,
       2026 - annee AS age
FROM film;
```

RÉSULTAT

| titre | age |
|---|---|
| La Haine | 31 |
| Les Visiteurs | 33 |
| Le Dîner de cons | 28 |
| Les Choristes | 22 |
| Intouchables | 15 |
| Le Grand Bain | 8 |

# Le filtre sur les lignes : WHERE

`= <> < <= > >=`
comparer deux valeurs

`AND OR NOT`
combiner des conditions, avec des parenthèses au besoin

`BETWEEN 1990 AND 1999`
un intervalle, bornes comprises

`IN (6, 10)`
une liste de valeurs possibles

`IS NULL`
une valeur absente ; jamais « = NULL »

```
SELECT titre, duree
FROM film
WHERE duree > 100;
```

→

| titre | duree |
|---|---|
| Les Visiteurs | 107 |
| Intouchables | 112 |
| Le Grand Bain | 122 |

```
SELECT titre, annee, duree
FROM film
WHERE annee >= 2000 AND duree < 120;
```

→

| titre | annee | duree |
|---|---|---|
| Les Choristes | 2004 | 97 |
| Intouchables | 2011 | 112 |

```
SELECT nom FROM abonne
WHERE telephone IS NULL;
```

→

| nom |
|---|
| Nguyen |

# Les motifs de texte : LIKE

```
SELECT titre
FROM film
WHERE titre LIKE 'Les%';
```

| | |
|---|---|
| `%` | n'importe quelle suite de caractères, même vide |
| `_` | exactement un caractère |

| MOTIF | SENS | TITRES TROUVÉS |
|---|---|---|
| `LIKE 'Les%'` | commence par « Les » | Les Visiteurs · Les Choristes |
| `LIKE '%s'` | finit par « s » | Les Visiteurs · Le Dîner de cons · Les Choristes · Intouchables |
| `LIKE '%in%'` | contient « in » | La Haine · Le Grand Bain |
| `LIKE '_a%'` | a un « a » en deuxième position | La Haine |
| `ILIKE '%in%'` | contient « in », majuscule ou non | La Haine · Intouchables · Le Grand Bain |

LIKE distingue majuscules et minuscules ; ILIKE, propre à PostgreSQL, les confond.

# Le tri et la coupe : ORDER BY, LIMIT

### Croissant, par défaut

```
SELECT titre, duree
FROM film
ORDER BY duree;
```

RÉSULTAT

| titre | duree |
|---|---|
| Le Dîner de cons | 80 |
| Les Choristes | 97 |
| La Haine | 98 |
| Les Visiteurs | 107 |
| Intouchables | 112 |
| Le Grand Bain | 122 |

### Décroissant, puis coupé

```
SELECT titre, duree
FROM film
ORDER BY duree DESC
LIMIT 2;
```

RÉSULTAT

| titre | duree |
|---|---|
| Le Grand Bain | 122 |
| Intouchables | 112 |

**ASC**, croissant, est l'ordre par défaut ; **DESC** l'inverse.

Plusieurs critères, par ordre de priorité : `ORDER BY prix DESC, date_seance`

Sans ORDER BY, l'ordre des lignes n'est pas garanti.

LIMIT s'applique après le tri : il garde les premières lignes triées.

# La jointure : réunir deux tables

```
SELECT s.id_seance, s.date_seance, f.titre
FROM seance s
JOIN film f ON f.id_film = s.id_film;
```

ON apparie chaque séance au film de même id_film : clé étrangère = clé primaire.

Les alias s et f précisent la table de chaque colonne.

Sans séance, Le Grand Bain est absent du résultat.

**seance s**

| id_seance | date_seance | id_film |
|---|---|---|
| 1 | 2026-11-02 | 1 |
| 2 | 2026-11-02 | 3 |
| 3 | 2026-11-04 | 5 |
| 4 | 2026-11-06 | 1 |
| 5 | 2026-11-07 | 5 |
| 6 | 2026-11-08 | 2 |
| 7 | 2026-11-09 | 4 |

**film f**

| id_film | titre |
|---|---|
| 1 | La Haine |
| 2 | Les Visiteurs |
| 3 | Le Dîner de cons |
| 4 | Les Choristes |
| 5 | Intouchables |
| 6 | Le Grand Bain |

→

**RÉSULTAT**

| id_seance | date_seance | titre |
|---|---|---|
| 1 | 2026-11-02 | La Haine |
| 2 | 2026-11-02 | Le Dîner de cons |
| 3 | 2026-11-04 | Intouchables |
| 4 | 2026-11-06 | La Haine |
| 5 | 2026-11-07 | Intouchables |
| 6 | 2026-11-08 | Les Visiteurs |
| 7 | 2026-11-09 | Les Choristes |

Pour : Pâtisserie 3(f), 3(g)

# Les jointures en chaîne

```
SELECT a.nom, f.titre, r.nb_places
FROM reservation r
JOIN abonne a ON a.id_abonne = r.id_abonne
JOIN seance s ON s.id_seance = r.id_seance
JOIN film f ON f.id_film = s.id_film
WHERE f.titre = 'La Haine'
ORDER BY a.nom;
```

RÉSULTAT

| nom | titre | nb_places |
|---|---|---|
| Benali | La Haine | 2 |
| Dupont | La Haine | 1 |
| Nguyen | La Haine | 1 |

Chaque ligne du résultat réunit des colonnes venues de tables différentes.




Quatre tables reliées, trois conditions ON : on avance de table en table en suivant les clés étrangères.

# Les fonctions d'agrégation

Un agrégat résume un ensemble de lignes en une seule valeur.

| | |
|---|---|
| `COUNT(*)` | le nombre de lignes |
| `COUNT(colonne)` | le nombre de valeurs non NULL |
| `SUM(expr)` | la somme |
| `AVG(expr)` | la moyenne |
| `MIN(expr)` | la plus petite valeur |
| `MAX(expr)` | la plus grande valeur |

Un agrégat peut porter sur un calcul : ci-contre, la somme des nb_places × prix.

```
SELECT COUNT(*) AS nb_abonnes,
       COUNT(telephone) AS nb_telephones
FROM abonne;
```

| nb_abonnes | nb_telephones |
|---|---|
| 4 | 3 |

```
SELECT SUM(r.nb_places * s.prix) AS recette
FROM reservation r
JOIN seance s ON s.id_seance = r.id_seance;
```

| recette |
|---|
| 194.50 |

# Le regroupement : GROUP BY

```
SELECT id_film, COUNT(*) AS nb_seances
FROM seance
GROUP BY id_film
ORDER BY id_film;
```

Les lignes de même id_film forment un groupe ; chaque groupe donne une ligne de résultat.

Dans SELECT : seulement les colonnes du GROUP BY et des agrégats.

Pour afficher le titre : joindre film, puis GROUP BY f.id_film, f.titre.

**seance**

| id_seance | id_film |
|---|---|
| 1 | 1 |
| 4 | 1 |
| 6 | 2 |
| 2 | 3 |
| 7 | 4 |
| 3 | 5 |
| 5 | 5 |

→

**RÉSULTAT**

| id_film | nb_seances |
|---|---|
| 1 | 2 |
| 2 | 1 |
| 3 | 1 |
| 4 | 1 |
| 5 | 2 |

Les séances sont rangées ici par id_film pour faire apparaître les cinq groupes, un par bande.

Pour : Pâtisserie 3(e), 3(f)

# Le filtre sur les groupes : HAVING

### Les films projetés plus d'une fois

```
SELECT f.titre, COUNT(*) AS nb_seances
FROM film f
JOIN seance s ON s.id_film = f.id_film
GROUP BY f.id_film, f.titre
HAVING COUNT(*) > 1;
```

| titre | nb_seances |
|---|---|
| Intouchables | 2 |
| La Haine | 2 |

### Les séances dont la recette dépasse 30

```
SELECT r.id_seance,
       SUM(r.nb_places * s.prix) AS recette
FROM reservation r
JOIN seance s ON s.id_seance = r.id_seance
GROUP BY r.id_seance
HAVING SUM(r.nb_places * s.prix) > 30;
```

| id_seance | recette |
|---|---|
| 3 | 42.50 |
| 5 | 70.00 |
| 2 | 36.00 |

WHERE filtre les lignes ; HAVING filtre les groupes, en répétant l'agrégat et non son alias.

Pour : Pâtisserie 3(g), 3(h)

# « Le plus… » : trier, puis couper

```sql
-- Le film qui a rapporté le plus
SELECT f.titre,
       SUM(r.nb_places * s.prix) AS recette
FROM film f
JOIN seance s ON s.id_film = f.id_film
JOIN reservation r ON r.id_seance = s.id_seance
GROUP BY f.id_film, f.titre
ORDER BY recette DESC
LIMIT 1;
```

AVANT LIMIT 1

| titre | recette |
|---|---|
| Intouchables | 112.50 |
| Le Dîner de cons | 36.00 |
| La Haine | 34.00 |
| Les Visiteurs | 12.00 |

ORDER BY … DESC place le plus grand en tête ; LIMIT 1 ne garde que cette première ligne.

En cas d'égalité, LIMIT 1 ne montre qu'un seul des ex æquo.

---

Variante par sous-requête, quand le maximum porte sur une seule colonne :

```sql
SELECT titre, duree FROM film WHERE duree = (SELECT MAX(duree) FROM film);
```

# « Jamais… » : LEFT JOIN ou NOT IN

### Avec LEFT JOIN et IS NULL

```sql
SELECT f.titre
FROM film f
LEFT JOIN seance s ON s.id_film = f.id_film
WHERE s.id_seance IS NULL;
```

### Avec NOT IN et une sous-requête

```sql
SELECT titre
FROM film
WHERE id_film NOT IN (
    SELECT id_film FROM seance
);
```

LEFT JOIN garde tous les films ; pour un film sans séance, les colonnes de seance valent NULL.

| titre | id_seance |
|---|---|
| La Haine | 1 |
| Le Dîner de cons | 2 |
| Intouchables | 3 |
| La Haine | 4 |
| Intouchables | 5 |
| Les Visiteurs | 6 |
| Les Choristes | 7 |
| Le Grand Bain | NULL |

WHERE … IS NULL ne retient que la dernière ligne. Les deux requêtes répondent : Le Grand Bain.

AIDE-MÉMOIRE

Pour : les quatre exercices

# Les messages d'erreur à connaître

| POSTGRESQL RÉPOND | CAUSE LA PLUS FRÉQUENTE |
|---|---|
| `column "La Haine" does not exist` | Texte entre guillemets doubles : mettre des apostrophes. |
| `syntax error at or near ")"` | Faute juste avant le mot cité : souvent une virgule en trop. |
| `relation "film" does not exist` | Table pas encore créée, ou nom mal orthographié. |
| `multiple primary keys for table … are not allowed` | Deux PRIMARY KEY : déclarer la clé composée en fin de table. |
| `… violates foreign key constraint …` | Parent pas encore inséré, ou supprimé avant ses enfants. |
| `duplicate key value violates unique constraint …` | Deux lignes ont la même clé primaire, ou la même valeur UNIQUE. |
| `column "f.titre" must appear in the GROUP BY clause …` | Colonne du SELECT absente du GROUP BY et hors agrégat. |
| `aggregate functions are not allowed in WHERE` | Condition sur un agrégat : elle va dans HAVING. |
| `column reference "id_film" is ambiguous` | Colonne présente dans deux tables : la préfixer (s.id_film). |

Lire le message jusqu'au bout : il cite presque toujours le nom ou le mot en cause.

AIDE-MÉMOIRE

Pour : Pâtisserie, question 3

# Du français au SQL

| QUAND L'ÉNONCÉ DIT | ON ÉCRIT |
|---|---|
| « quels sont les… » | `SELECT … FROM …` |
| « dont la valeur est supérieure à… » | `WHERE colonne > valeur` |
| « dont le nom commence par… » | `WHERE colonne LIKE 'X%'` |
| « triés par… » | `ORDER BY colonne` |
| des informations réparties dans plusieurs tables | `JOIN … ON clé étrangère = clé primaire` |
| « combien », « le total », « la moyenne » | `COUNT(*), SUM(…), AVG(…)` |
| « pour chaque… », « par… » | `GROUP BY colonne` |
| « ayant plus de n… » | `GROUP BY … HAVING COUNT(*) > n` |
| « le plus… », « celui qui a le plus… » | `ORDER BY … DESC LIMIT 1` |
| « jamais », « aucun » | `LEFT JOIN … WHERE … IS NULL` |